FEU INSTITUTE OF TECHNOLOGY

COLLEGE OF COMPUTER STUDIES

CS054
(INTELLIGENT SYSTEMS)

EXERCISE
3

Data Mining

Group Name:	Christian Dave K. Salvador
Members:	**Name** / **Role** (empty)
Section:	AN42
Professor:	Ms. Crisola G. Tan

I. PROGRAM OUTCOME/S (PO) ADDRESSED BY THE LABORATORY EXERCISE

- Design and evaluate solutions for computing problems. [PO: e]

II. COURSE LEARNING OUTCOME/S (CLO) ADDRESSED BY THE LABORATORY EXERCISE

- To design an Intelligent System based on methods learned. [CLO: 3]

III. INTENDED LEARNING OUTCOME/S (ILO) OF THE LABORATORY EXERCISE

At the end of this exercise, students must be able to:

- Demonstrate understanding of the following algorithms: Decision Tree, K-Mean, Apriori.
- Implement Decision Tree Algorithm by designing and developing a simple predictor program using Python.

IV. BACKGROUND INFORMATION

In order to accomplish this exercise, the student must have a clear understanding of the following topics:

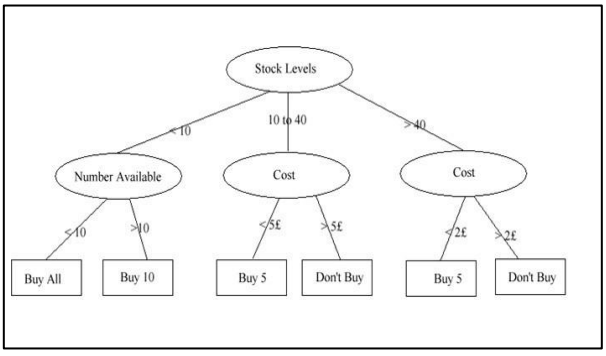
Decision Tree Algorithm

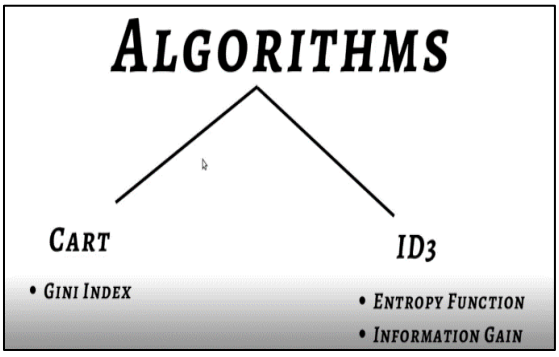
- Related to most of the other techniques (primarily classification and prediction), the decision tree can be used either as a part of the selection criteria, or to support the use and selection of specific data within the overall structure.

A Decision Tree is a tree:

- where each node represents a feature (attribute)
- each branch represents a decision (rule)
- each leaf represents an outcome

Example of Decision Tree





https://octaviansima.wordpress.com/2011/03/25/decision-trees-c4-5/

What is an ID3 Algorithm?

- Stands for Iterative Dichotomiser 3
- An algorithm developed by J.Ross Quinlan
- The core algorithm for building decision trees.
- Supervised learning algorithm used for classification problems.
- It is a classification algorithm that follows a greedy approach by selecting a best attribute that yields maximum Information Gain(IG) or minimum Entropy(H).

STEPS in Creating the Decision Tree
Important Concepts:

- Calculate **Entropy (the amount of uncertainty in dataset : the number of positive and negative evidences).**

$$Entropy = \frac{-p}{p+n} log_2(\frac{p}{p+n}) - \frac{n}{p+n} log_2(\frac{n}{p+n})$$

** Entropy(S) – the entropy of the entire data set.
** Computing the entropy of each attribute

- Calculate **Average Information**.

$$I(Attribute) = \sum \frac{p_i + n_i}{p+n} Entropy(A)$$

- Calculate **Information Gain** (Difference in Entropy before and after splitting dataset on attribute A)

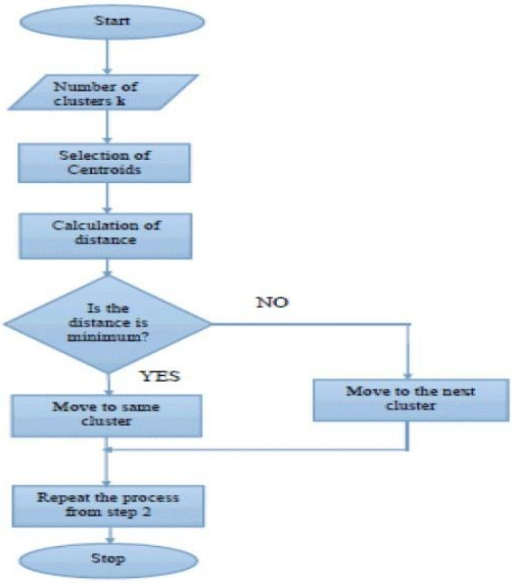
$$Gain = Entropy(S) - I(Attribute)$$

K-Means Algorithm

K-means creates k groups from a set of objects so that the members of a group are more similar. It's a popular cluster analysis technique for exploring a dataset.
Cluster analysis is a family of algorithms designed to form groups such that the group members are more similar versus non-group members. Clusters and groups are synonymous in the world of cluster analysis.

K-means Clustering Algorithm

1. Choose a value of k number of clusters to be formed.
2. Randomly select k data points from the data set as the initial cluster centroids/centers
3. For each datapoint:
   Compute the distance between thedatapoint and the cluster centroid
   Assign the datapoint to the closest centroid
4. 4. For each cluster calculate the new mean based on the datapoints in the cluster.
5. 5. Repeat 3 & 4 steps until mean of the clusters stops changing or maximum number of iterations reached.



https://miro.medium.com/max/1022/1*ZN6cDw34wEO-dxydDwY4vA.png

Apriori Algorithm

What does it do?
It learns association rules and is applied to a database containing a large number of transactions.

What is Association Rule?
It is a data mining technique for learning correlations and relations among variables in a database.

Apriori – Example
In a database full of supermarket transactions, think of a database as a giant spreadsheet where each row is a customer transaction and every column represents a different grocery item.

Transaction ID	Chips	Dip	Soda	Apples	Milk
1	X	X	X		
2	X	X			X
3	X		X		

By applying the Apriori algorithm, we can learn the grocery items that are purchased together a.k.a association rule. One can find those items that tend to be purchased together more frequently than other items — the ultimate goal being to get shoppers to buy more. Together, these items are called “itemsets”.

How Apriori works?
Define first the size of itemset. Do you want to see patterns for a 2-itemset, etc.?
The second is support or the number of transactions containing the itemset divided by the total number of transactions. An itemset that meets the support is called a frequent itemset.

The third is your confidence or the conditional probability of some item, given you have certain other items in your itemset.
A good example is, given chips is in your itemset, there is a 67% confidence of having soda also in the itemset.

V. GRADING SYSTEM / RUBRIC (please see separate sheet)

VI. LABORATORY ACTIVITY

In this problem, you will come up with a decision tree to predict if a planet is habitable based only on features observed by the telescope.

In the table below, you are given the data from all 800 planets surveyed so far. The features observed by telescope are Size ("Big" or "Small"), and Orbit ("Near" or "Far"). Each row indicates the values of the features and habitability, and how many times that set of values was observed. So, for example, there were 20 "Big" planets "Near" their star that were habitable.

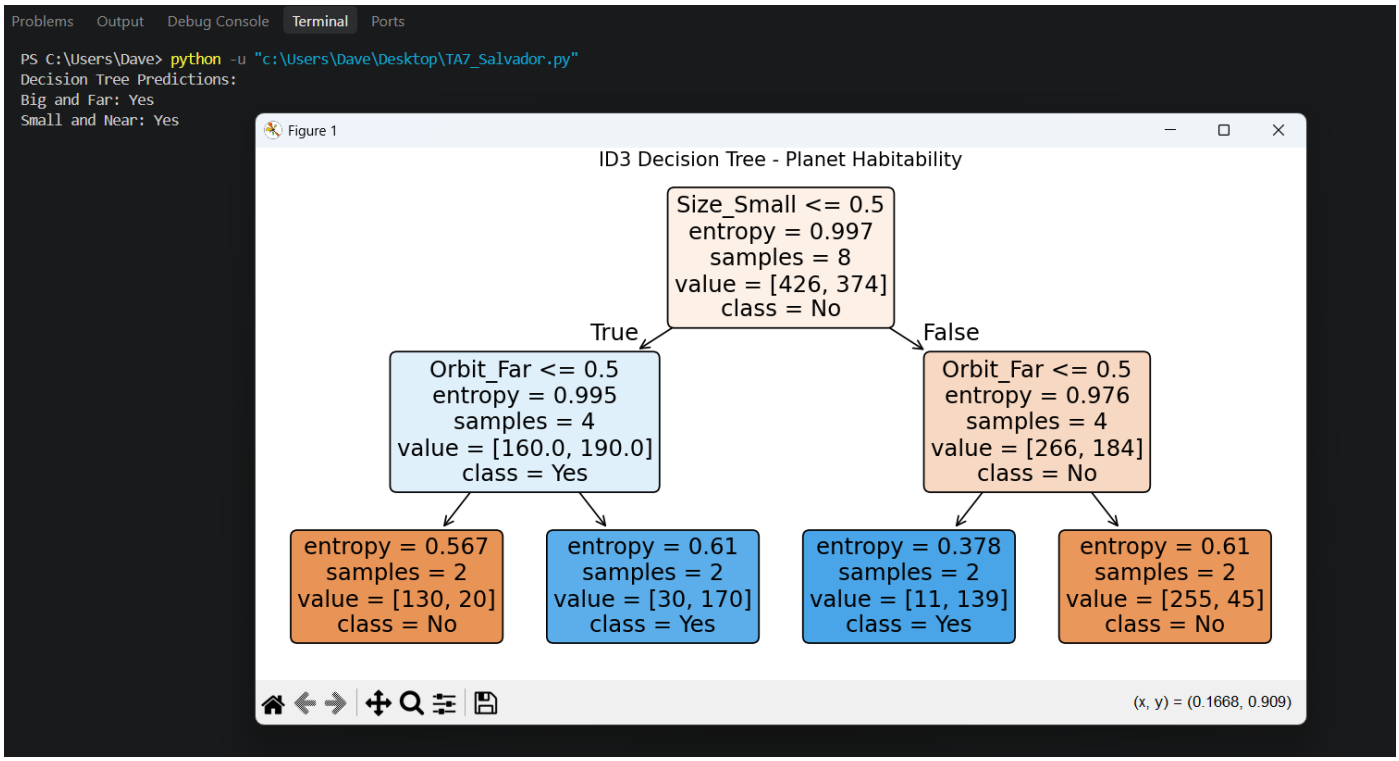
Size	Orbit	Habitable	Count
Big	Near	Yes	20
Big	Far	Yes	170
Small	Near	Yes	139
Small	Far	Yes	45
Big	Near	No	130
Big	Far	No	30
Small	Near	No	11
Small	Far	No	255

a. Determine if Planet=Big and Orbit=Far is habitable? Derive and draw the decision tree learned by ID3 on this data.

b. Determine if Planet=Small and Orbit=Near is habitable? Derive and draw the decision tree learned by ID3 on this data.

Derive a decision tree learned by ID3 on this data (use the maximum information gain criterion for splits, don't do any pruning). Make sure to clearly mark at each node what attribute you are splitting on, and which value corresponds to which branch. By each leaf node of the tree, write in the number of habitable and inhabitable planets in the training data that belong to that node.

OUTPUT:


Problems
Output
Debug Console
Terminal
Ports
PS C:\Users\Dave> python -u "c:\Users\Dave\Desktop\TA7_Salvador.py"
Decision Tree Predictions:
Big and Far: Yes
Small and Near: Yes
Figure 1
ID3 Decision Tree - Planet Habitability
Size_Small <= 0.5
entropy = 0.997
samples = 8
value = [426, 374]
class = No
True
False
Orbit_Far <= 0.5
entropy = 0.995
samples = 4
value = [160.0, 190.0]
class = Yes
Orbit_Far <= 0.5
entropy = 0.976
samples = 4
value = [266, 184]
class = No
entropy = 0.567
samples = 2
value = [130, 20]
class = No
entropy = 0.61
samples = 2
value = [30, 170]
class = Yes
entropy = 0.378
samples = 2
value = [11, 139]
class = Yes
entropy = 0.61
samples = 2
value = [255, 45]
class = No
(x, y) = (0.1668, 0.909)

CODE:

```
from sklearn.tree import DecisionTreeClassifier, plot_tree
import matplotlib.pyplot as plt
import pandas as pd

# Dataset from Laboratory Activity 3
data = pd.DataFrame({
    "Size": ["Big", "Big", "Small", "Small",
             "Big", "Big", "Small", "Small"],
    "Orbit": ["Near", "Far", "Near", "Far",
              "Near", "Far", "Near", "Far"],
    "Habitable": ["Yes", "Yes", "Yes", "Yes",
                  "No", "No", "No", "No"],
    "Count": [20, 170, 139, 45, 130, 30, 11, 255]
})

# Convert categorical values into numbers
X = pd.get_dummies(data[["Size", "Orbit"]], dtype=int)
y = data["Habitable"]

# Train ID3-style decision tree using entropy
model = DecisionTreeClassifier(
    criterion="entropy",
    random_state=0
)

model.fit(X, y, sample_weight=data["Count"])

# Predict the two requested planets
planets = pd.DataFrame({
    "Size": ["Big", "Small"],
    "Orbit": ["Far", "Near"]
})

planets_encoded = pd.get_dummies(
    planets, dtype=int
```

```
).reindex(columns=X.columns, fill_value=0)

predictions = model.predict(planets_encoded)

print("Decision Tree Predictions:")
print("Big and Far:", predictions[0])
print("Small and Near:", predictions[1])

# Draw the decision tree
plt.figure(figsize=(12, 7))
plot_tree(
    model,
    feature_names=X.columns,
    class_names=model.classes_,
    filled=True,
    rounded=True,
    proportion=False
)
plt.title("ID3 Decision Tree - Planet Habitability")
plt.tight_layout()
plt.show()
```

CODE SCREENSHOT:

TA7_Salvador.py

C: > Users > Dave > Desktop > TA7_Salvador.py > ...

```python
from sklearn.tree import DecisionTreeClassifier, plot_tree
import matplotlib.pyplot as plt
import pandas as pd

# Dataset from Laboratory Activity 3
data = pd.DataFrame({
    "Size": ["Big", "Big", "Small", "Small",
             "Big", "Big", "Small", "Small"],
    "Orbit": ["Near", "Far", "Near", "Far",
              "Near", "Far", "Near", "Far"],
    "Habitable": ["Yes", "Yes", "Yes", "Yes",
                  "No", "No", "No", "No"],
    "Count": [20, 170, 139, 45, 130, 30, 11, 255]
})

# Convert categorical values into numbers
X = pd.get_dummies(data[["Size", "Orbit"]], dtype=int)
y = data["Habitable"]

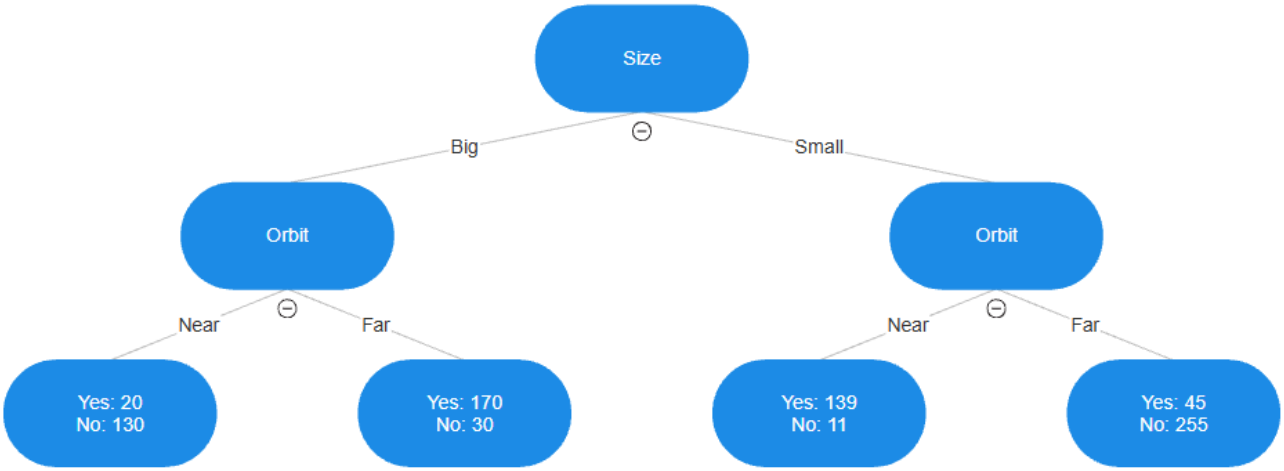
# Train ID3-style decision tree using entropy
model = DecisionTreeClassifier(
    criterion="entropy",
    random_state=0
)

model.fit(X, y, sample_weight=data["Count"])

# Predict the two requested planets
planets = pd.DataFrame({
    "Size": ["Big", "Small"],
    "Orbit": ["Far", "Near"]
})
```

```
33
34  planets_encoded = pd.get_dummies(
35      planets, dtype=int
36  ).reindex(columns=X.columns, fill_value=0)
37
38  predictions = model.predict(planets_encoded)
39
40  print("Decision Tree Predictions:")
41  print("Big and Far:", predictions[0])
42  print("Small and Near:", predictions[1])
43
44  # Draw the decision tree
45  plt.figure(figsize=(12, 7))
46  plot_tree(
47      model,
48      feature_names=X.columns,
49      class_names=model.classes_,
50      filled=True,
51      rounded=True,
52      proportion=False
53  )
54  plt.title("ID3 Decision Tree - Planet Habitability")
55  plt.tight_layout()
56  plt.show()
57
```

DECISION TREE SCREENSHOT (DRAW):



VII. QUESTION AND ANSWER

Given the details below, apply K-means algorithm to generate iteration 2 and 3. Show your solution.

We will apply k-means on the following 1 dimensional data set for K=2.
Data set {2, 4, 10, 12, 3, 20, 30, 11, 25}

Datapoint	D1	D2	Cluster
2	2	9	C1
4	0	7	C1
10	6	1	C2
12	8	1	C2
3	1	8	C1
20	16	9	C2
30	26	19	C2
11	7	0	C2
25	21	14	C2

Iteration 1
M1, M2 are the two randomly selected centroids/means where
M1= 4, M2=11
and the initial clusters are
C1= {4}, C2= {11}

OUTPUT:

```
Problems  Output  Debug Console  Terminal  Ports

PS C:\Users\Dave> python -u "c:\Users\Dave\Desktop\TA7-Part 2_Salvador.py"

Iteration 2
C1 = [2, 4, 3]
C2 = [10, 12, 20, 30, 11, 25]
M1 = 3.0
M2 = 18.0

Iteration 3
C1 = [2, 4, 10, 3]
C2 = [12, 20, 30, 11, 25]
M1 = 4.75
M2 = 19.6
PS C:\Users\Dave>
```

CODE:

```
# K-Means Clustering for 1D Data
data = [2, 4, 10, 12, 3, 20, 30, 11, 25]

# Initial centroids
m1 = 4
m2 = 11

for iteration in range(2, 4):
    # Assign points to the nearest centroid
    c1 = []
    c2 = []

    for point in data:
        if abs(point - m1) <= abs(point - m2):
            c1.append(point)
        else:
            c2.append(point)

    # Calculate new centroids
    new_m1 = sum(c1) / len(c1)
    new_m2 = sum(c2) / len(c2)

    print(f"\nIteration {iteration}")
    print("C1 =", c1)
    print("C2 =", c2)
    print("M1 =", new_m1)
    print("M2 =", new_m2)

    # Update centroids
    m1 = new_m1
    m2 = new_m2
```

CODE SCREENSHOT:

TA7_Salvador.py ● TA7-Part 2_Salvador.py ×

C: > Users > Dave > Desktop > TA7-Part 2_Salvador.py > ...

```python
# K-Means Clustering for 1D Data
data = [2, 4, 10, 12, 3, 20, 30, 11, 25]

# Initial centroids
m1 = 4
m2 = 11

for iteration in range(2, 4):
    # Assign points to the nearest centroid
    c1 = []
    c2 = []

    for point in data:
        if abs(point - m1) <= abs(point - m2):
            c1.append(point)
        else:
            c2.append(point)

    # Calculate new centroids
    new_m1 = sum(c1) / len(c1)
    new_m2 = sum(c2) / len(c2)

    print(f"\nIteration {iteration}")
    print("C1 =", c1)
    print("C2 =", c2)
    print("M1 =", new_m1)
    print("M2 =", new_m2)

    # Update centroids
    m1 = new_m1
    m2 = new_m2

```

VIII. REFERENCES

- Decision Trees: Some Exercises
  Exemplifying the application of the ID3 algorithm in the presence of both categorical and continue attributes, CMU, 2012 fall, Eric Xing, Aarti Singh, HW1, pr. 1.1
- Li, R.. Top 10 Data Mining Algorithms, Hacker Bits. Retrieved from https://hackerbits.com/data/k-means-data-mining-algorithm/
- Shamil, F.R., 3-Mar-2020. Support, Confidence, Minimum support, Frequent itemset, K-itemset, absolute support in data mining. T4 Tutorials. Retrieved from https://t4tutorials.com/support-confidence-minimum-support-frequent-itemset-in-data-mining/
- Schott, M., 23-April-2019. K-Means Clustering Algorithm for Machine Learning. Retrieved from https://medium.com/capital-one-tech/k-means-clustering-algorithm-for-machine-learning-d1d7dc5de882